

STA 445 S26 Assignment 4

2026-03-17

Load all the packages you will use in this chunk. Notice, I have set `include = FALSE` because I don't want to see you loading packages in the output.

Problem 1

For the following regular expression, explain in words what it matches on. Then add test strings to demonstrate that it in fact does match on the pattern you claim it does. Do at least 4 tests. Make sure that your test set of strings has several examples that match as well as several that do not. Make sure to remove the `eval=FALSE` so that the code and output shows in your pdf.

- a. This regular expression matches: *Insert your answer here...In this example, the regular expression matches a string that contains the letter a, basically not mattering where the a is placed. So if there is at least one a in the expression it will return TRUE otherwise FALSE*

```
strings <- c("maple", "ambient", "log", "aim", "bird")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, 'a') )
```

```
##    string result
## 1  maple    TRUE
## 2 ambient    TRUE
## 3   log    FALSE
## 4   aim     TRUE
## 5  bird    FALSE
```

- b. This regular expression matches: *If the expression shows a string that contains a and b one after another. A needs to be right after the b in order for the pattern matches. If there is not a sequence that contains a and then a b right after then it will return a FALSE*

```
strings <- c("pablo", "travel", "case", "polish", "table")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, 'ab') )
```

```
##    string result
## 1  pablo    TRUE
## 2 travel    FALSE
## 3   case    FALSE
## 4 polish    FALSE
## 5  table    TRUE
```

- c. This regular expression matches: *For this expression there is a match when there is either an a or a b and it will print the output TRUE. These square brackets say that this means either can be present. Otherwise if none appear it will return FALSE*

```
strings <- c("dable", "front", "airplane", "brick", "prior")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '[ab]') )
```

```
##      string result
## 1    dable    TRUE
## 2   front   FALSE
## 3 airplane   TRUE
## 4    brick    TRUE
## 5   prior   FALSE
```

- d. This regular expression matches: *This means that the expression has to start with either an A or B and the symbol ^ means it needs to be at the start of the string. If A or B is present at the beginning of the string then it will return TRUE otherwise FALSE.*

```
strings <- c("brake", "steak", "alternate", "cable", "baseline")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '^[ab]') )
```

```
##      string result
## 1    brake    TRUE
## 2    steak   FALSE
## 3 alternate   TRUE
## 4    cable   FALSE
## 5 baseline   TRUE
```

- e. This regular expression matches: *This means that the expression can contain any digit possible and that it will include a space after the digit followed by either an uppercase A or lowercase a. This will return TRUE otherwise it will be FALSE.*

```
strings <- c("4 A", "210b", "794 a", "1 a", "77 d", "52100 A")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '\\d+\\s[aA]') )
```

```
##      string result
## 1     4 A    TRUE
## 2    210b   FALSE
## 3    794 a    TRUE
## 4     1 a    TRUE
## 5    77 d   FALSE
## 6 52100 A    TRUE
```

- f. This regular expression matches: *This expression means that again any digit is allowed or more and that 0 spaces are followed by or as many spaces as possible. If this happens then its TRUE if not then its FALSE*

```
strings <- c("60 a", "40A", "500 c", "2 a", "432b")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '\\d+\\s*[aA]') )
```

```
##      string result
## 1 60 a TRUE
## 2 40A TRUE
## 3 500 c FALSE
## 4 2 a TRUE
## 5 432b FALSE
```

- g. This regular expression matches: *The dot matches any character (letters, numbers, spaces, symbols) The asterisk means zero or more of that character Thus the regular expression makes a match of zero or more characters and since it allows zero matches, the empty string is a match This means that any string will match this regular expression*

```
strings <- c("", "34 a", "safd", "Hello", "90 9 333")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '.*') )
```

```
##      string result
## 1 TRUE
## 2 34 a TRUE
## 3 safd TRUE
## 4 Hello TRUE
## 5 90 9 333 TRUE
```

- h. This regular expression matches: *With a similar description, this expression matches any string that starts with exactly two word characters, followed by bar The caret character ^ matches the beginning of the string The part \\w{2} matches exactly two word characters, where a word character is any letter, digit, or underscore*

```
strings <- c("labar", "77bar", "opbar", "3333bar", "ka bar")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '^\\w{2}bar') )
```

```
##      string result
## 1 labar TRUE
## 2 77bar TRUE
## 3 opbar TRUE
## 4 3333bar FALSE
## 5 ka bar FALSE
```

- i. This regular expression matches: *This regular expression has two alternatives, as indicated by the OR operator. The first matches the string “foo bar” literally, as well as the literal dot character. The second matches any string that begins with exactly two word characters (letters, digits, or underscores) followed by the string “bar” at the beginning of the string.*

```
strings <- c("foo.bar", "ararbar", "pabar", "foi.bar", "__bar")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '(foo\\.bar)|(^\\w{2}bar)') )
```

```
##      string result
## 1 foo.bar   TRUE
## 2 ararbar  FALSE
## 3 pabar    TRUE
## 4 foi.bar  FALSE
## 5 __bar    TRUE
```

Problem 2

The following file names were used in a camera trap study. The S number represents the site, P is the plot within a site, C is the camera number within the plot, the first string of numbers is the YearMonthDay and the second string of numbers is the HourMinuteSecond.

```
file.names <- c( 'S123.P2.C10_20120621_213422.jpg',
                  'S10.P1.C1_20120622_050148.jpg',
                  'S187.P2.C2_20120702_023501.jpg')
```

Produce a data frame with columns corresponding to the **site**, **plot**, **camera**, **year**, **month**, **day**, **hour**, **minute**, and **second** for these three file names. So we want to produce code that will create the data frame:

Site	Plot	Camera	Year	Month	Day	Hour	Minute	Second
S123	P2	C10	2012	06	21	21	34	22
S10	P1	C1	2012	06	22	05	01	48
S187	P2	C2	2012	07	02	02	35	01

```
split_file_name <- str_split(file.names, "\\.")
```

```
Site <- sapply(split_file_name, `[`, 1)
Plot <- sapply(split_file_name, `[`, 2)
Rest <- sapply(split_file_name, `[`, 3)
split_rest <- str_split(Rest, "_")
```

```
Camera <- sapply(split_rest, `[`, 1)
Date <- sapply(split_rest, `[`, 2)
Time <- sapply(split_rest, `[`, 3)
```

```
Year <- str_sub(Date, 1, 4)
Month <- str_sub(Date, 5, 6)
```

```
Day <- str_sub(Date, 7, 8)

Hour <- str_sub(Time, 1, 2)
Minute <- str_sub(Time, 3, 4)
Second <- str_sub(Time, 5, 6)
data.frame(Site, Plot, Camera, Year, Month, Day, Hour, Minute, Second)
```

```
##   Site Plot Camera Year Month Day Hour Minute Second
## 1 S123   P2    C10 2012    06  21   21     34     22
## 2 S10    P1     C1 2012    06  22   05     01     48
## 3 S187   P2     C2 2012    07  02   02     35     01
```

3. The full text from Lincoln's Gettysburg Address is given below. Calculate the mean word length
Note: consider 'battle-field' as one word with 11 letters).

```
Gettysburg <- 'Four score and seven years ago our fathers brought forth on this continent, a new
```

```
new_clean_text <- str_replace_all(Gettysburg, "[^a-zA-Z\\- ]", "")
all_words <- unlist(str_split(new_clean_text, " "))

words_list <- all_words[all_words != ""]
word_length <- str_length(words_list)
mean(word_length)
```

```
## [1] 4.18705
```